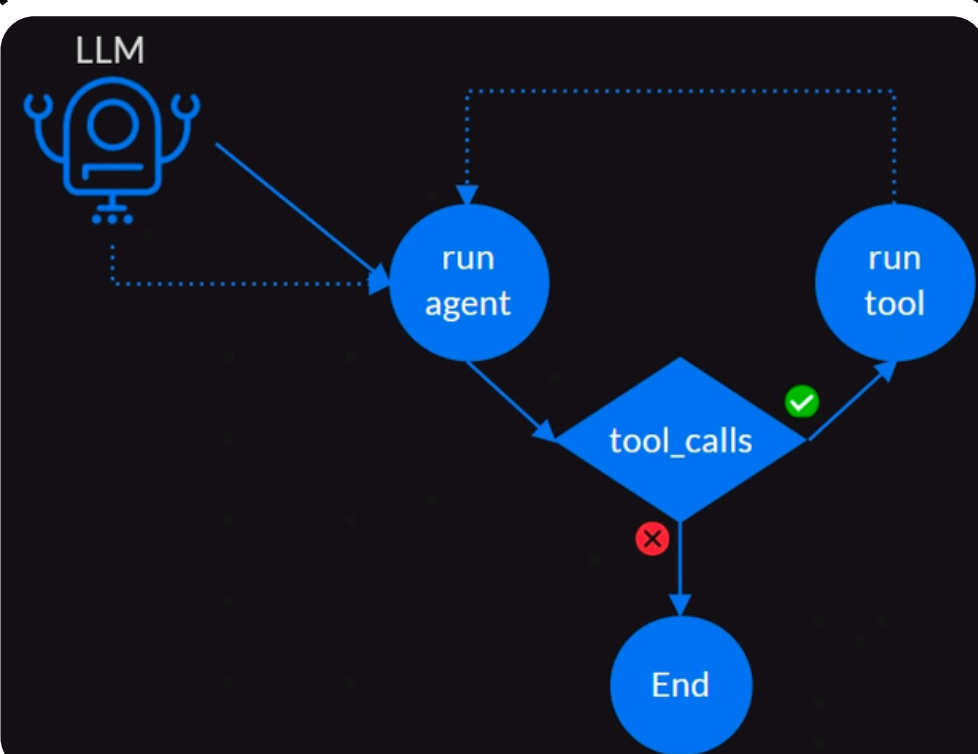
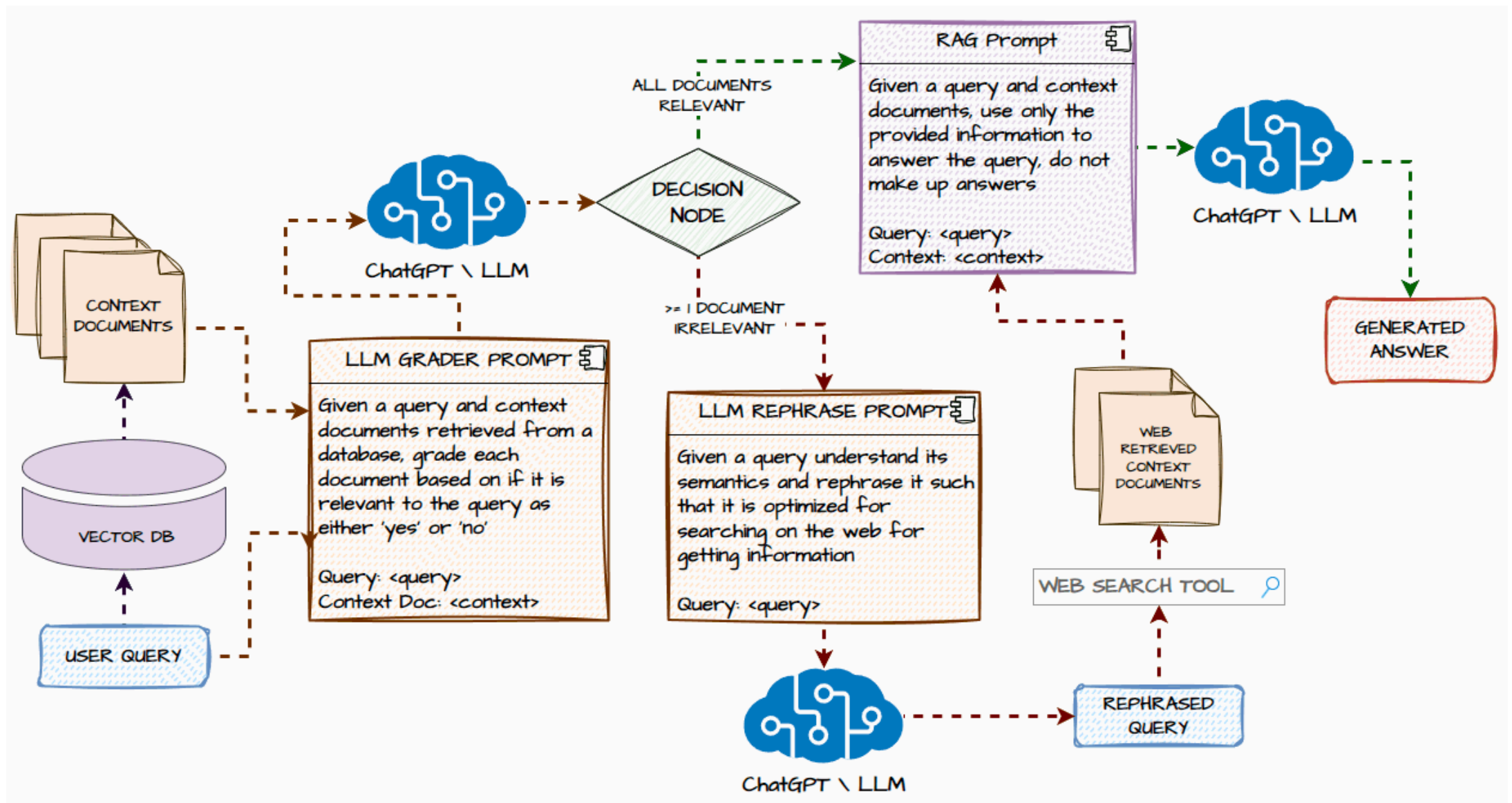


Day 20 of Mastering AI Agents

A Guide to Agentic RAG



- Step 1 - Retrieve Node
 - Retrieves context documents from vector database from input query
- Step 2 - Grade Node
 - Use an LLM to grade if retrieved documents are relevant to input question - yes or no
- Step 3A - Generate Answer Node
 - If all documents are relevant (all 'yes'), send it to an LLM for response generation

- Step 3B - Rewrite Query Node
 - If some or all documents are not relevant (at least one 'no'), rephrase the query
- Step 3C - Web Search Node
 - Search the web to get context information using rephrased query
- Step 3D - Generate Answer Node
 - Send rephrased query and context documents or information to the LLM for response generation

Traditional RAG System Limitations

Traditional RAG systems have several limitations, some of which are mentioned as follows:

- They are not privy to real-time data
- The system is as good as the data you have in your vector database
- A bad retrieval strategy can lead to irrelevant documents being used to answer questions
- LLM can be prone to hallucinations or not being able to answer questions

In this post, we will focus particularly on the limitations of the RAG system, which does not have access to real-time data, as well as make sure the retrieved document chunks are actually relevant to answer the question. This will allow the RAG system to answer questions on more recent events and real-time data and be less prone to hallucinations.

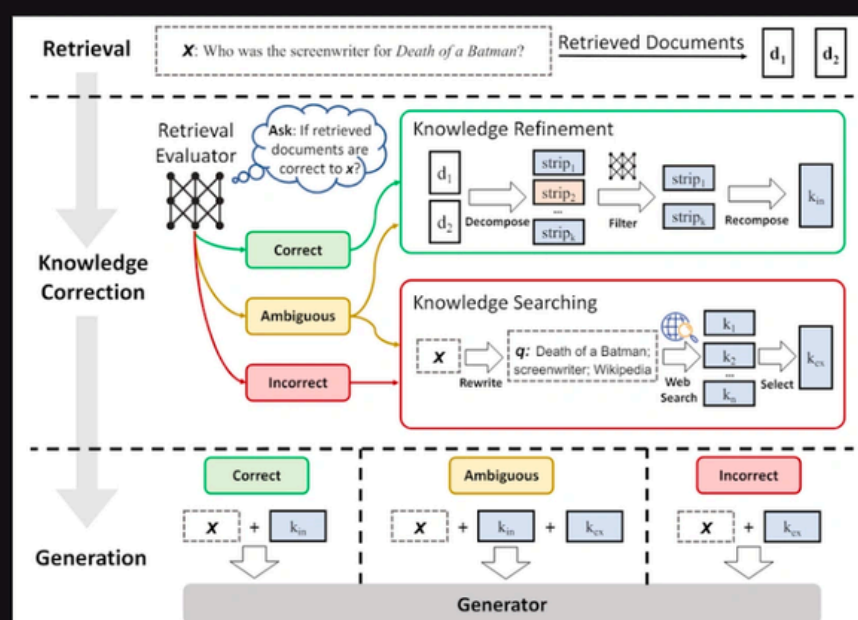


Corrective RAG System

The inspiration for our agentic RAG system will be based on the solution proposed in the paper, Corrective Retrieval Augmented Generation, Yan et al. , where they propose a workflow as depicted in the following figure to enhance a regular RAG system. The key idea here is to retrieve document chunks from the vector database as usual and then use an LLM to check if each retrieved document chunk is relevant to the input question.

If all the retrieved document chunks are relevant, then it goes to the LLM for a normal response generation like a standard RAG pipeline. However, suppose some retrieved documents are not relevant to the input question. In that case, we rephrase the input query, search the web to retrieve new information related to the input question, and then send it to the LLM to generate a response.

Corrective RAG Workflow proposed in the CRAG paper



- **Step 1:**
 - Retrieve context documents from vector database from the input query
- **Step 2:**
 - Use an LLM to check if retrieved documents are relevant to input question
- **Step 3:**
 - If all documents are relevant (Correct), no specific action needed
- **Step 4:**
 - If some or all documents are not relevant (Ambiguous OR Incorrect), rephrase the query and search the web to get relevant context information
- **Step 5:**
 - Send rephrased query and context documents or information to the LLM for response generation

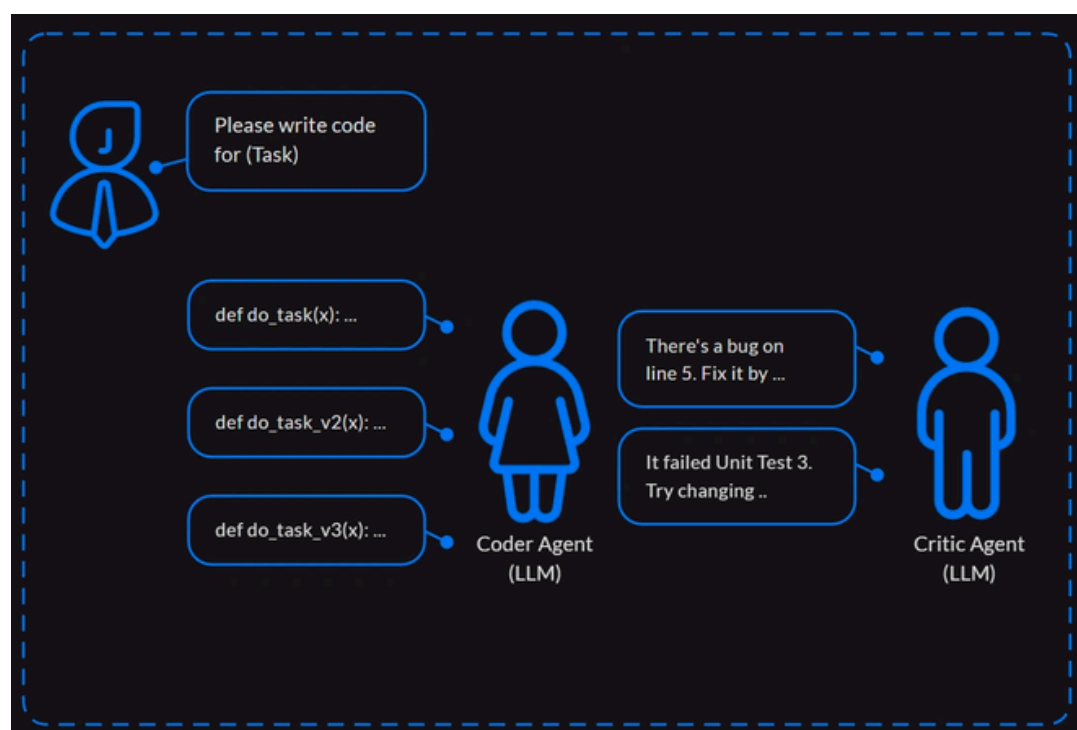


The key novelty in this approach involves searching the web, augmenting static information in the vector database with live and real-time data, and checking whether retrieved documents are relevant to the input question—something that simple embedding cosine similarity cannot capture.

The Rise of AI Agents

AI Agents or Agentic AI systems have seen a rise, especially in 2024, which enables us to build Generative AI systems that can reason, analyze, interact, and take actions automatically. The whole idea of Agentic AI is to build completely autonomous systems that can understand and manage complex workflows and tasks with minimal human intervention. Agentic systems can grasp nuanced concepts, set and pursue goals, reason through tasks, and adapt their actions based on changing conditions. These systems can consist of a single agent or multiple agents.

The example below shows two agents working together to transform the user's instructions into functioning code snippets.

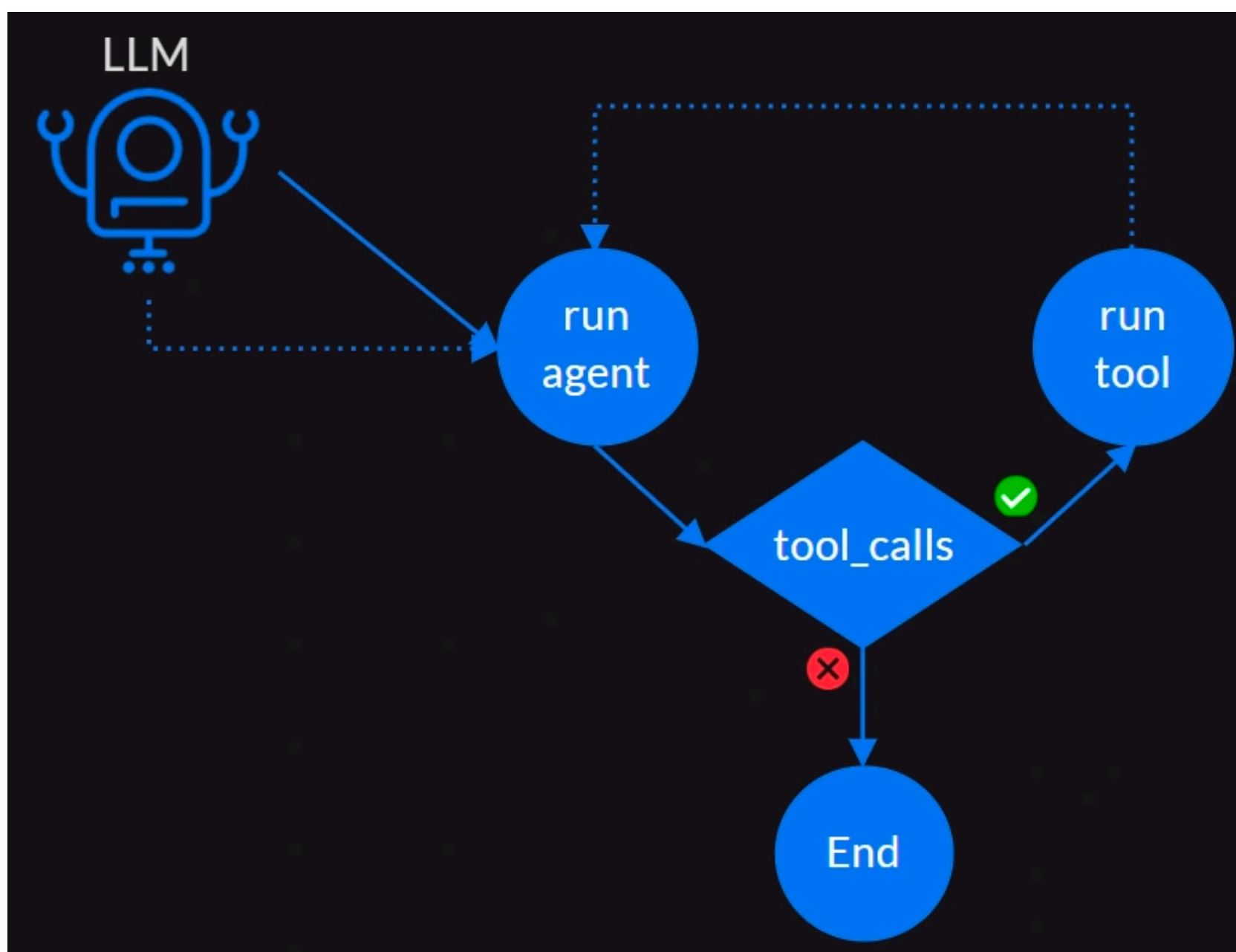


One can use various frameworks to build Agentic AI systems, including CrewAI, LangChain, LangGraph, AutoGen, and many more.



Using these frameworks enables us to develop complex workflows with ease. Remember, an agent is basically one or more LLMs having access to a set of tools that they can leverage based on specific prompt-based instructions to answer user questions.

We will be using LangGraph for our practical implementation of our Agentic RAG system. LangGraph, built on top of LangChain, facilitates the creation of cyclical graphs essential for developing AI agents powered by LLMs. The widely-used NetworkX library inspires its interface. It enables the coordination and checkpointing of multiple chains (or actors) through cyclic computational steps. LangGraph treats Agent workflows as a cyclical Graph structure, as depicted in the following figure.



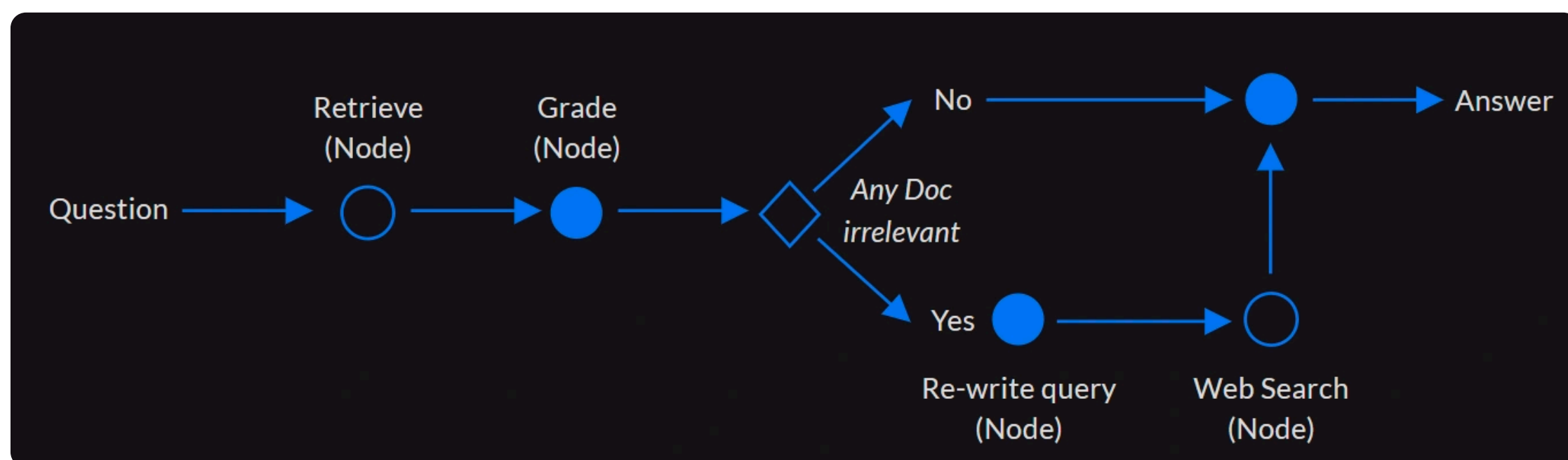
The main components in any LangGraph agent include:

- **Nodes:** Functions or LangChain Runnable objects such as tools
- **Edges:** Specify directional paths between nodes
- **Stateful Graphs:** Manage and update state objects while processing data through nodes

LangGraph leverages this to facilitate cyclical LLM call executions with state persistence, which AI agents often require.

Agentic Corrective RAG System Workflow

In this section, we will see a high-level workflow of the main components in our Agentic RAG system and the execution flow among these components. The following figure illustrates this in detail.



Each component in this workflow is represented by a node. There are two major flows here in this Agentic RAG system. One flow is the regular RAG system workflow, where we have a user question and retrieve context documents from the vector database. However, we introduce an additional step here based on the corrective RAG paper where we use an LLM to check if all retrieved documents are relevant to the user question (in the grade node); if they are all relevant, then we generate a response using an LLM as shown in the following snapshot.

- **Step 1 - Retrieve Node**
 - Retrieves context documents from vector database from input query
- **Step 2 - Grade Node**
 - Use an LLM to grade if retrieved documents are relevant to input question - yes or no
- **Step 3A - Generate Answer Node**
 - If all documents are relevant (all 'yes'), send it to an LLM for response generation

The other flow occurs in case at least one or more of the retrieved context documents from the vector database are irrelevant to the user question, as depicted in the following snapshot. Then, we leverage an LLM to rewrite the user query and optimize it for search on the web. Next, we leverage a web search tool to search the web using this rephrased query and get some new documents. Finally, we send the query and any relevant context documents (including the web search documents) to an LLM to generate a response.



- **Step 3B -Rewrite Query Node**
 - If some or all documents are not relevant (at least one 'no'), rephrase the query
- **Step 3C - Web Search Node**
 - Search the web to get context information using rephrased query
- **Step 3D - Generate Answer Node**
 - Send rephrased query and context documents or information to the LLM for response generation

For more information, kindly visit this [article](#)

Advanced

Artificial Intelligence

Generative AI

Guide

Large Language Models

LLMs

RAG

A Comprehensive Guide to Building Agentic RAG Systems with LangGraph

Guide on LangGraph optimizes Agentic RAG Systems with advanced contextual intelligence, offering robust solutions to enhance Gen AI assistant

Dipanjan (DJ) Sarkar 04 Apr, 2025

